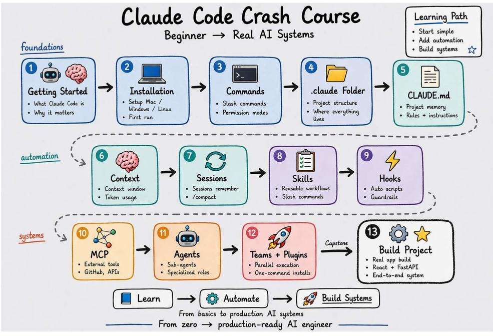
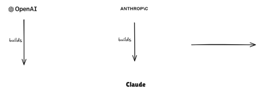
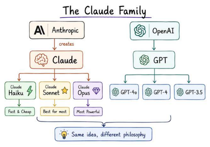
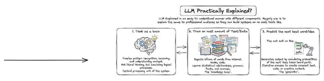
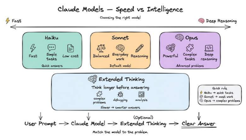
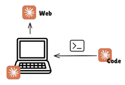

Claude Code Crash Course
Beginner → Real AI Systems
Learning Path
• Start simple
• Add automation
• Build systems
foundations
1 Getting Started
• What Claude Code is
• Why it matters
2 Installation
• Setup Mac / Windows / Linux
• First run
3 Commands
• Slash commands
• Permission modes
4 .claude Folder
• Project structure
• Where everything lives
5 CLAUDE.md
• Project memory
• Rules + instructions
automation
6 Context
• Context window
• Token usage
7 Sessions
• Sessions remember
• /compact
8 Skills
• Reusable workflows
• Slash commands
9 Hooks
• Auto scripts
• Guardrails
systems
10 MCP
• External tools
• GitHub, APIs
11 Agents
• Sub-agents
• Specialized roles
12 Teams + Plugins
• Parallel execution
• One-command installs
Capstone
13 Build Project
• Real app build
• React + FastAPI
• End-to-end system
Learn → Automate → Build Systems
From basics to production AI systems
From zero → production-ready AI engineer

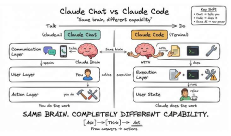

OpenAI
builds
ANTHROPIC
builds
Claude

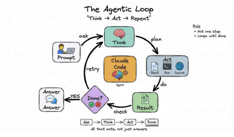

The Claude Family
Anthropic
creates
Claude
Claude Haiku
Claude Sonnet
Claude Opus
Fast & Cheap
Best for most
Most Powerful
OpenAI
GPT
GPT-4o
GPT-4
GPT-3.5
Same idea, different philosophy


LLM Practically Explained?
1. Think as a brain
2. Train on vast amount of Text/Data
3. Predict the next best word/idea
The cat sat on the ...

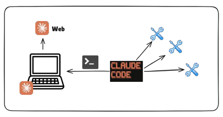

Claude Models — Speed vs Intelligence
Choosing the right model
Fast
Deep Reasoning
Haiku
Fast
Simple tasks
Low cost
Quick answers
Sonnet
Balanced
Everyday work
Reasoning
Default model
Opus
Powerful
Complex tasks
Deep reasoning
Advanced problems
Extended Thinking
Think longer before answering
complex problems
debugging
analysis
Slower → smarter answers
Quick rule
Haiku → quick tasks
Sonnet → most work
Opus → complex problems
User Prompt → Claude Model → (Optional) Extended Thinking → Clear Answer
Match the model to the problem.

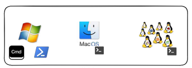



Claude Chat vs Claude Code
"Same brain, different capability"
Key Shift
• Chat → tells you
• Code → does it
• Same AI → new power
Talk
Do
(claude.ai) Claude Chat
Claude Code (Terminal)
Communication Layer
talks
Claude Brain
Same brain
acts
WITH
speaks
does
User Layer
You
advice
execution
Execution Layer
Action Layer
you do
runs
User State
relax
You do the work
Claude does the work
SAME BRAIN. COMPLETELY DIFFERENT CAPABILITY.
[Ask] → [Think] → Act
From answers → actions


The Agentic Loop
"Think → Act → Repeat"
Rule
• Not one step
• Loops until done
Prompt
ask
Think
plan
Act
Read
Run
Search
do
Result
check
Done?
YES
Answer
Answer
retry
Claude Code
Agent
Ask → Think → Act → Done
AI that acts, not just answers

SETUP & INSTALLATION

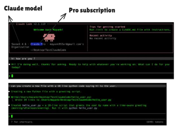

Web
CLAUDE CODE



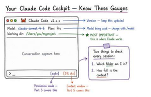
All OS

Claude Desktop

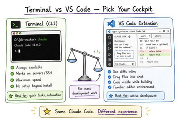
Claude doctor

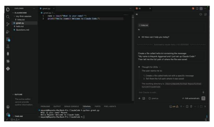

Claude model
Pro subscription

```
Claude Code v2.1.119
            Welcome back Mayank!                Tips for getting started
                                                Run /init to create a CLAUDE.md file with instructions…
                                                Recent activity
                                                No recent activity
Sonnet 4.6 · Claude Pro · mayank953a1@gmail.com's
Organization
          ~/Desktop/TestClaudeCode

 hi how are you ?

● Hi! I'm doing well, thanks for asking. Ready to help with whatever you're working on. What can I do for you
  today?

>

 can you create a new file with a 10 line python code saying hi to the user.

● Creating a new Python file with a greeting script.

● Write(/Users/mayank/Desktop/TestClaudeCode/hello_user.py)
  ⎿ Wrote 14 lines to /Users/mayank/Desktop/TestClaudeCode/hello_user.py

● Created hello_user.py — a 10-line script that greets the user by name with a time-aware greeting
  (morning/afternoon/evening). Run it with python hello_user.py.

>
? for shortcuts                                                          16991 tokens
```


Your Claude Code Cockpit — Know These Gauges
Claude Code v2.x.x
Version — keep this updated
Model: claude-sonnet-4-6 Plan: Pro
Model being used — change with /model
Working dir: /Users/you/myproject
MOST IMPORTANT — this is where Claude works
Conversation appears here
Two things to check every session:
1. Which folder am I in?
2. How full is the context?
[auto]
[5% ctx]
Permission mode — Part 3 covers this
Context window — Part 5 covers this

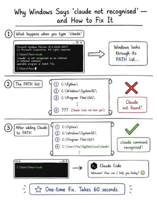
Claude Code Terminal UI Overview


Terminal vs VS Code — Pick Your Cockpit
Terminal (CLI)
C:\job-tracker> claude
Claude Code v1.0.0
Always available
Works on servers/SSH
Maximum speed
No setup beyond install
Best for: quick tasks, automation
For most development work
VS Code Extension
See diffs inline
Drag files into chat
Code visible while building
Familiar editor environment
Best for: active development
Same Claude Code. Different experience.



Create a file called hello.txt containing the message: "My name is Mayank Aggarwal and I just set up Claude Code." Then tell me the full path of where the file was saved.

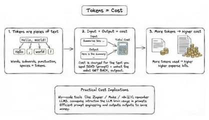
Claude Code Extension UI Overview

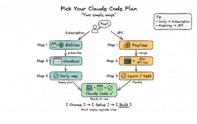

Why Windows Says 'claude not recognised' — and How to Fix It
1 What happens when you type 'claude'
Windows looks through its PATH list...
2 The PATH list
C:\Python\
C:\Windows\System32\
C:\Program Files\Git\
??? (Claude Code not here yet!)
Claude not found!
3 After adding Claude to PATH
C:\Python\
C:\Windows\System32\
C:\Program Files\Git\
C:\Users\You\AppData\Local\claude\
claude command recognised!
C:\Users\You>claude
Claude Code
Welcome! How can I help you today?
One-time fix. Takes 60 seconds.

```
Microsoft Windows [Version 10.0.22631.3447]
(c) Microsoft Corporation. All rights reserved.
C:\Users\You>claude
'claude' is not recognized as an internal
or external command,
operable program or batch file.
C:\Users\You>
```

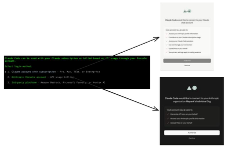
Installing and Path Issues

Token = 3/4th of a word

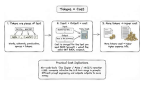

Tokens = Cost
1. Tokens are pieces of text
Hello, world!
Hello
,
world
!
Words, subwords, punctuation, spaces = tokens.
2. Input + Output = cost
Input
Output
Total Cost
3. More tokens → higher cost
More tokens used → higher higher expense bills.
Practical Cost Implications


Pick Your Claude Code Plan
"Two simple ways"
Pay?
Tip
• Daily → Subscription
• Exploring → API
Subscription
API
Step 1 $20/mo
subscribe
Step 2 claude.ai
Step 3 Daily use
Simple plan
Step 1 Pay/use
usage
Step 2 API key
Step 3 Learn / test
Flexible
Claude Code ✓
Ready to use
[Choose] → [Setup] → [Build]
Start simple, upgrade later

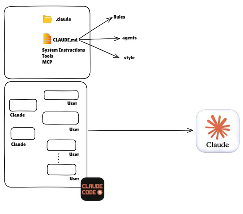
Claude doctor

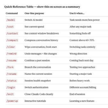


```
Claude Code can be used with your Claude subscription or billed based on API usage through your Console
account.

Select login method:

❯ 1. Claude account with subscription · Pro, Max, Team, or Enterprise

  2. Anthropic Console account · API usage billing

  3. 3rd-party platform · Amazon Bedrock, Microsoft Foundry, or Vertex AI
```


Tokens = Cost

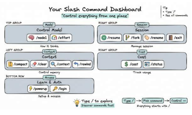
Sessions, Commands, Modes
& the .Claude Directory


.claude
CLAUDE.md
System Instructions
Tools
MCP
Rules
agents
style
Claude
User
Claude
User
User
User
Claude
CLAUDE CODE



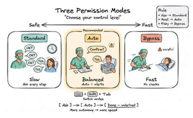
Quick Reference Table — show this on screen as a summary

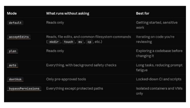
Command	One-line purpose	Use it when...
`/model`	Switch AI model	Task needs more/less power
`/cost`	See current spend	After any major task
`/context`	See context window breakdown	Something feels off
`/compact`	Compress conversation history	Context above 60-70%
`/clear`	Wipe conversation, fresh start	Switching tasks entirely
`/rewind`	Undo messages + file changes	Wrong direction
`/resume`	Continue a past session	Coming back next day
`/fork`	Branch the conversation	Testing two approaches
`/rename`	Name the current session	Starting a major task
`/status`	Session health snapshot	Before heavy work
`/login`	Switch authentication	Different account/billing
`/exit`	Close Claude Code cleanly	End of session
`/powerup`	Interactive tutorials	Learning a new feature


Your Slash Command Dashboard
"Control everything from one place"
Tip
• Type '/'
• See all commands
TOP GROUP
Model
Control Model
/model
/effort
How it thinks
RIGHT GROUP
Session
Session
/resume
/fork
/rename
/exit
Manage session
LEFT GROUP
Context
Context
/compact
/clear
/context
/rewind
Control memory
RIGHT GROUP
Cost
Cost
/cost
/status
Track usage
BOTTOM ROW
Access
Learn & Auth
/powerup
/login
Setup & access
Type / to explore
Discover commands fast
Type / → Pick command → Control —
Everything starts with /

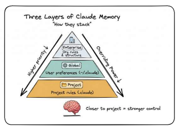
Compact

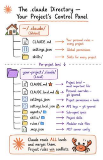
https://claude.ai/public/artifacts/532827c1-ba83-42f2-b0b1-5113d7f8babe

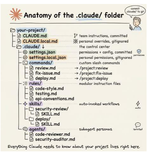

Three Permission Modes
"Choose your control level"
Safe
Fast
Rule
• New → Standard
• Most → Auto
• Risky → Bypass
Standard
OK?
OK?
OK?
Yes
Yes
Slow
Ask every step
Recommended
Auto
Confirm?
Yes
Balanced
Auto + alerts
Bypass
careful
Fast
No checks
+ Shift + Tab
Switch modes
[Ask] → [Auto] → [Done – underlined]
More autonomy → more speed



Mode	What runs without asking	Best for
`default`	Reads only	Getting started, sensitive work
`acceptEdits`	Reads, file edits, and common filesystem commands (`mkdir` , `touch` , `mv` , `cp` , etc.)	Iterating on code you're reviewing
`plan`	Reads only	Exploring a codebase before changing it
`auto`	Everything, with background safety checks	Long tasks, reducing prompt fatigue
`dontAsk`	Only pre-approved tools	Locked-down CI and scripts
`bypassPermissions`	Everything except protected paths	Isolated containers and VMs only

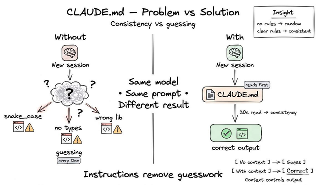

Three Layers of Claude Memory
"How they stack"
Enterprise
Org rules & structure
Global
User preferences (~/.claude)
Project
Project rules (.claude)
Higher priority
Overriding Power
Closer to project = stronger control


The .claude Directory —
Your Project's Control Panel
~/.claude/
(Global)
CLAUDE.md
Your personal rules – every project
settings.json
Global permissions
skills/
Skills for every project
Per-project level
your-project/.claude/
(Local)
CLAUDE.md
Project brief – most important file
CLAUDE.local.md
Personal overrides – git ignored
settings.json
Project permissions + hooks
settings.local.json
API keys – git ignored
agents/
Sub-agent specs
skills/
Project skills
rules/
Modular rule files
.mcp.json
MCP server config
Claude reads ALL levels and merges them.
Project rules win conflicts.

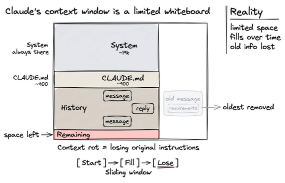

Anatomy of the .claude/ folder
commit .claude/ to git
your-project/
CLAUDE.md
team instructions, committed
CLAUDE.local.md
personal overrides, gitignored
.claude/
the control center
settings.json
permissions + config, committed
settings.local.json
personal permissions, gitignored
permissions
commands/
custom slash commands
review.md
→ /project:review
fix-issue.md
→ /project:fix-issue
deploy.md
→ /project:deploy
rules/
modular instruction files
code-style.md
testing.md
api-conventions.md
skills/
auto-invoked workflows
auto
security-review/
SKILL.md
deploy/
SKILL.md
agents/
subagent personas
isolated
code-reviewer.md
security-auditor.md
Everything Claude needs to know about your project lives right here.

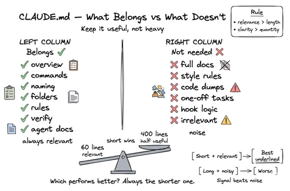
.claude **https://code.claude.com/docs/en/claude-directory#ce-claude-md**

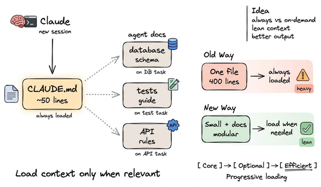
The Anatomy of CLAUDE.md


CLAUDE.md — Problem vs Solution
Consistency vs guessing
Insight
no rules → random
clear rules → consistent
Without
New session
snake_case
no types
wrong lib
guessing
every time
Same model
• Same prompt •
Different result
With
New session
reads first
CLAUDE.md
30s read → consistency
correct output
Instructions remove guesswork
[No context] → [Guess]
[With context] → [Correct]
Context controls output

<system-reminder>
IMPORTANT: this context may or may not be relevant to your tasks.
You should not respond to this context unless it is highly relevant to your task.
</system-reminder>


Claude's context window is a limited whiteboard
Reality
limited space
fills over time
old info lost
System
always there
System
~19k
CLAUDE.md
~900
CLAUDE.md
~900
History
message
reply
message
old message
requirements
oldest removed
space left
Remaining
Context rot = losing original instructions
[Start] → [Fill] → [Lose]
Sliding window


CLAUDE.md — What Belongs vs What Doesn't
Keep it useful, not heavy
Rule
• relevance > length
• clarity > quantity
LEFT COLUMN
Belongs
overview
commands
naming
folders
rules
verify
agent docs
always relevant
RIGHT COLUMN
Not needed
full docs
style rules
code dumps
one-off tasks
hook logic
irrelevant
noise
60 lines relevant
short wins
400 lines half useful
[Short + relevant] → [Best underlined]
[Long + noisy] → [Worse]
Signal beats noise
Which performs better? Always the shorter one.


Claude
new session
CLAUDE.md
~50 lines
always loaded
agent docs
database schema
on DB task
tests guide
on test task
API rules
on API task
Idea
always vs on-demand
lean context
better output
Old Way
One file 400 lines
always loaded
heavy
New Way
Small + docs modular
load when needed
lean
[Core] → [Optional] → [Efficient]
Progressive loading
Load context only when relevant

CLAUDE.md is not a one and done kind of file.

It should also be regularly maintained.

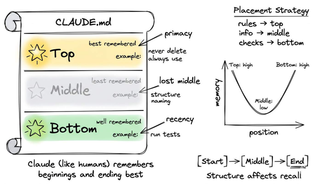
1. Review Monthly

2. Prune Aggresively

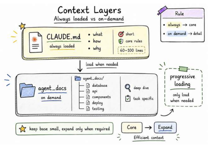
3. Don't include instructions for solved problems.

4. Prefer pointers over copies.

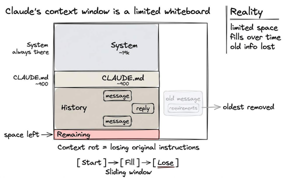
[From V2, use this framing]
"I treat CLAUDE.md like technical debt and reduce it over time, not grow it."

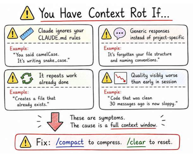

CLAUDE.md
Top
best remembered
example:
primacy
never delete
always use
Middle
least remembered
example:
lost middle
structure
naming
Bottom
well remembered
example:
recency
run tests
Claude (like humans) remembers beginnings and ending best
Placement Strategy
rules → top
info → middle
checks → bottom
memory
position
Top: high
Bottom: high
Middle: low
[Start] → [Middle] → [End]
Structure affects recall

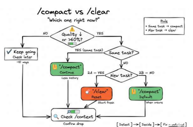
Primacy and Recency Bias

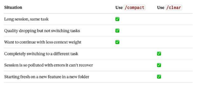

Context Layers
Always loaded vs on-demand
Rule
• always → core
• on demand → detail
CLAUDE.md
always loaded
• what
• how
• why
short
core rules
60–100 lines
load when needed
agent_docs
on demand
agent_docs/
database
api
components
deploy
testing
deep dive
task specific
progressive loading
only load when needed
keep base small, expand only when required
Core
Expand
Efficient context

Context Management (Rot)

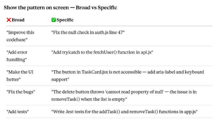

Claude's context window is a limited whiteboard
Reality
limited space
fills over time
old info lost
System
always there
System
~19k
CLAUDE.md
~900
CLAUDE.md
~900
History
message
reply
message
old message
requirements
oldest removed
space left
Remaining
Context rot = losing original instructions
[Start] → [Fill] → [Lose]
Sliding window


You Have Context Rot If...
Claude ignores your CLAUDE.md rules
Example:
"You said camelCase. It's writing snake_case."
Generic responses instead of project-specific
Example:
It's forgotten your file structure and naming conventions."
It repeats work already done
Example:
"Creates a file that already exists."
Quality visibly worse than early in session
Example:
"Code that was clean 30 messages ago is now sloppy."
These are symptoms.
The cause is a full context window.
Fix: /compact to compress. /clear to reset.


/compact vs /clear
"Which one right now?"
Rule
• Same task → compact
• New task → clear
Quality ↓ or >60%?
NO
YES
Keep going
Check later
~10 msgs
Same task?
YES (same task)
NO
"/compact"
Continue
Less history
New task?
2A – YES
2B – NO
"/clear"
Reset
Start fresh
"/compact"
Default
When unsure
Check /context
Confirm drop
Manage context smartly
[Detect] → [Decide] → [Fix – underlined]
Manage context smartly

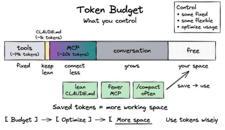


Situation	Use `/compact`	Use `/clear`
Long session, same task	✅	
Quality dropping but not switching tasks	✅	
Want to continue with less context weight	✅	
Completely switching to a different task		✅
Session is so polluted with errors it can't recover		✅
Starting fresh on a new feature in a new folder		✅



Show the pattern on screen — Broad vs Specific

❌ Broad	✅ Specific
"Improve this codebase"	"Fix the null check in auth.js line 47"
"Add error handling"	"Add try/catch to the fetchUser() function in api.js"
"Make the UI better"	"The button in TaskCard.jsx is not accessible — add aria-label and keyboard support"
"Fix the bugs"	"The delete button throws 'cannot read property of null' — the issue is in removeTask() when the list is empty"
"Add tests"	"Write Jest tests for the addTask() and removeTask() functions in app.js"


Token Budget
What you control
Control
• some fixed
• some flexible
• optimize usage
CLAUDE.md (~1k tokens)
tools (~19k tokens)
MCP (~20k tokens)
conversation
free
fixed
keep lean
connect less
grows
your space
lean CLAUDE.md
fewer MCP
/compact often
save → use
Saved tokens = more working space
[Budget] → [Optimize] → [More space
Use tokens wisely

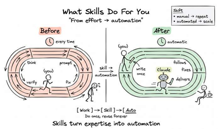
1. Keep Claude.md concise
2. Move instructions from Claude.md to skills, agents, settings.
3. MCP server overhead.

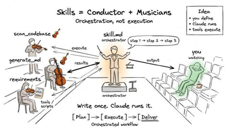
4. Extended Thinking.

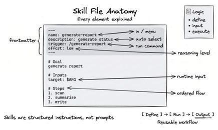
5. Plan before you Build

6. Please check your code and divide in version (V1)

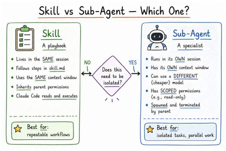
The Anatomy of SKILLS


What Skills Do For You
"From effort → automation"
Shift
• manual → repeat
• automated → scale
Before
every time
think
prompt
(you)
verify
fix
skill → automation
After
automatic
(you)
write once
Claude
follows
fixes
delivers
[Work] → [Skill] → [Auto
Do once, reuse forever
Skills turn expertise into automation

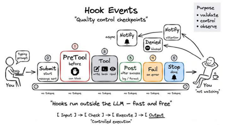

Skills = Conductor + Musicians
Orchestration, not execution
Idea
• you define
• Claude runs
• tools execute
scan_codebase
generate_md
requirements
skill.md
orchestrator
step 1 → step 2 → step 3
execute
results
output
you
watching
tools / scripts
orchestrator
Write once. Claude runs it.
[Plan] → [Execute] → [Deliver
Orchestrated workflow


Skill File Anatomy
Every element explained
Logic
• define
• input
• execute
frontmatter
in / menu
auto select
run command
reasoning level
runtime input
ordered flow
Skills are structured instructions, not prompts
[Define] → [Run] → [Output]
Reusable workflow

```
---
name: generate-report
description: generate status
trigger: /generate-report
effort: low
---

# Goal
generate report

# Inputs
target: $ARG

# Steps
1. scan
2. summarise
3. write
```


Skill vs Sub-Agent — Which One?
Skill
A playbook
• Lives in the SAME session
• Follows steps in skill.md
• Uses the SAME context window
• Inherits parent permissions
• Claude Code reads and executes
Best for:
repeatable workflows
NO
Does this need to be isolated?
YES
Sub-Agent
A specialist
• Runs in its OWN session
• Has its OWN context window
• Can use a DIFFERENT (cheaper) model
• Has SCOPED permissions (e.g., read-only)
• Spawned and terminated by parent
Best for:
isolated tasks, parallel work

HOOKS in CLAUDE CODE

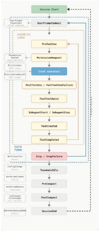

Hook Events
"Quality control checkpoints"
Purpose
• validate
• control
• observe
typing prompt
You
0 Submit start message sent
1 PreTool before can block
2 Tool write bash read
3 Post after success log / format
4 Fail on error
5 Stop done
async Notify
Denied blocked
Notify attention
You "not watching"
no tokens
"Hooks run outside the LLM — fast and free"
[Input] → [Check] → [Execute] → [Output
"Controlled execution"

1. Automation
2. Logging
3. Guardrails

CLAUDE CODE

event driven architecture

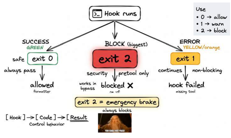
Think
thought of creating a file
actually creating a file – bash command
what to add in file
add the code in file
see the other files
narrow down a code
do some edits
delete some file
edit some mode files

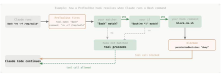

Session Start
UserPromptSubmit
UserPrompt Expansion
AGENTIC LOOP
PreToolUse
Permission Denied
PermissionRequest
Elicitation (MCP input)
[tool executes]
ElicitationResult (MCP input)
PostToolUse / PostToolUseFailure
PostToolBatch
SubagentStart / SubagentStop
TaskCreated
TaskCompleted
EACH TURN
Notification (Async)
Stop / StopFailure
ConfigChange (Async)
TeammateIdle
WorktreeCreate (Setup)
PreCompact
WorktreeRemove (Teardown)
PostCompact
CwdChanged FileChanged (Env reaction)
InstructionsLoaded (Async)
SessionEnd

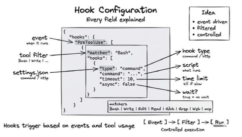
rm -rf

Creating a file

Exit code	Meaning	Use for
`0`	Success — proceed	Everything that should allow the action
`2`	BLOCK the action	Security guards in PreToolUse ONLY
`1`	Non-blocking error	Hook failed but don't stop Claude

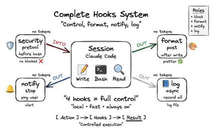

Hook runs
Use
• 0 → allow
• 1 → warn
• 2 → block
SUCCESS GREEN
safe
exit 0
always pass
allowed
formatter
BLOCK (biggest)
exit 2
security
pretool only
works in bypass
blocked ✕
rm -rf
ERROR YELLOW/orange
exit 1
continues
non-blocking
hook failed
missing tool
exit 2 = emergency brake
always blocks
[Hook] → [Code] → [Result
Control behavior
YOU SHALL NOT PASS!


Example: how a PreToolUse hook resolves when Claude runs a Bash command
Claude runs
Bash "rm -rf /tmp/build"
PreToolUse fires
tool_name: "Bash"
command: "rm -rf /tmp/build"
your matcher
"Bash" match?
yes
your if
"Bash(rm *)" match?
yes
your hook command
block-rm.sh
no
no
hook not matched
tool proceeds
blocked
permissionDecision: "deny"
tool call blocked
Claude Code continues
tool call allowed

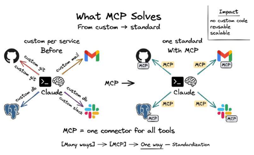

Hook Configuration
Every field explained
Idea
• event driven
• filtered
• controlled
event
when it runs
tool filter
Bash | Write | ...
settings.json
command / http
hook type
command / http
script
what runs
time limit
kill if slow
wait?
true = no wait
matchers
Bash | Write | Edit | Read | Glob | Grep | Web | mcp
Hooks trigger based on events and tool usage
[Event] → [Filter] → [Run]
Controlled execution

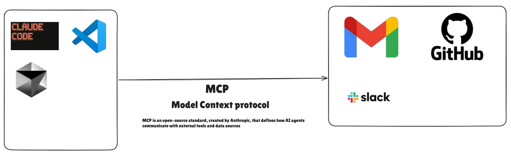
```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "...",
            "timeout": 10,
            "async": false
          }
        ]
      }
    ]
  }
}
```

Matcher	When it fires
`"Bash"`	Every terminal command Claude runs
`"Write"`	Every time Claude creates a new file
`"Edit"`	Every time Claude edits an existing file
`"Write|Edit"`	New files AND edits
`"Read"`	Every time Claude reads a file
(no matcher)	Every tool call of any type
`"mcp__github__.*"`	Any GitHub MCP tool call


Complete Hooks System
"Control, format, notify, log"
Roles
• block
• format
• notify
• log
no tokens
security
pretool
before bash
rm blocked ✕
INTO
Session
Claude Code
Write
Bash
Read
OUT
format
post
after write
prettier ✓
OUT
notify
stop
ping user
alert
OUT
log
async
record all
log file
"4 hooks = full control"
"local • fast • always on"
[Action] → [Hooks] → [Result]
"Controlled execution"

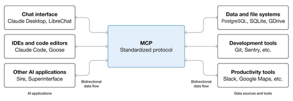
Model Context Protocol in Claude Code

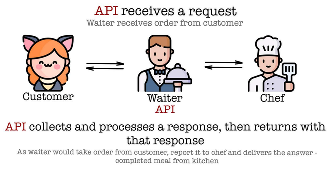

What MCP Solves
From custom → standard
Impact
no custom code
reusable
scalable
custom per service
Before
custom git
custom git
custom mail
custom db
custom db
custom slack
Claude
MCP →
one standard
With MCP
MCP
Claude
MCP = one connector for all tools
[Many ways] → [MCP] → One way — Standardization

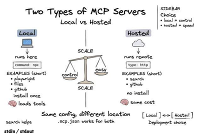

CLAUDE CODE
GitHub
slack

MCP
Model Context protocol

MCP is an open-source standard, created by Anthropic, that defines how AI agents communicate with external tools and data sources

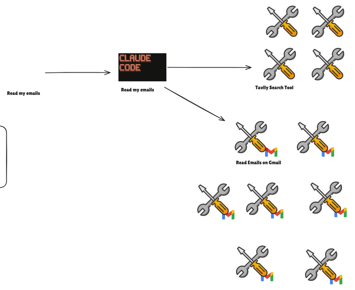

Chat interface
Claude Desktop, LibreChat
IDEs and code editors
Claude Code, Goose
Other AI applications
5ire, Superinterface
AI applications
Bidirectional data flow
MCP
Standardized protocol
Bidirectional data flow
Data and file systems
PostgreSQL, SQLite, GDrive
Development tools
Git, Sentry, etc.
Productivity tools
Slack, Google Maps, etc.
Data sources and tools


API receives a request
Waiter receives order from customer
Customer
Waiter
API
Chef
API collects and processes a response, then returns with that response
As waiter would take order from customer, report it to chef and delivers the answer - completed meal from kitchen


Two Types of MCP Servers
Local vs Hosted
SIDEBAR
Choice
• local = control
• hosted = speed
Local
runs here
command: npx
EXAMPLES (short)
• playwright
• files
• github
install once
loads tools
SCALE
control
easy
SCALE
Hosted
runs remote
type: http
EXAMPLES (short)
• search
• github
no install
same cost
Same config, different location
.mcp.json works for both
[Local] ↔ [Hosted]
Deployment choice

search helps

stdin / stdout


Read my emails
CLAUDE CODE
Read my emails
Tavily Search Tool
Read Emails on Gmail

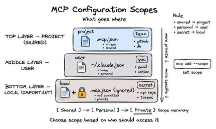
gmail – read
write
create draft
have an attachment

delete

archive

LABEL

Time


MCP Configuration Scopes
What goes where
Rule
• shared → project
• personal → user
• secret → local
TOP LAYER — PROJECT (SHARED)
project
.mcp.json
• in repo
• shared
team
• github
• db
MIDDLE LAYER — USER
user
~/.claude.json
• home
• personal
you
• email
• notion
BOTTOM LAYER — LOCAL (IMPORTANT)
local
.mcp.json (ignored)
• not committed
• private
secret
• api keys
• tokens
more private
more specific
mcp add --scope
set scope
[Shared] → [Personal] → [Private] Scope hierarchy
Choose scope based on who should access it

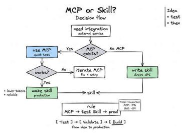
Connection Closed

```
Problem: npx doesn't execute directly on Windows
Fix: Add cmd /c wrapper in .mcp.json

Wrong:
"command": "npx",
"args": ["-y", "@package/name"]

Right:
"command": "cmd",
"args": ["/c", "npx", "-y", "@package/name"]
```

```
Problem: Slow startup (common with heavy Python packages)
Fix: Increase timeout with environment variable:
MCP_TIMEOUT=30000 claude   (30 seconds)

Or add to settings.json env field:
{
  "env": {
    "MCP_TIMEOUT": "30000"
  }
}
```

Server Disconnected
Auth Failed
Errors in Tools

Server Timeout Error

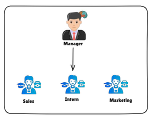

MCP or Skill?
Decision flow
Idea
• test fast
• then optimize
need integration
external service
MCP exists?
Yes
No MCP
use MCP
quick test
works?
No
iterate MCP
fix + retry
Yes
write skill
direct API
make skill
production
• lower tokens
• reliable
skill
rule
MCP → test Skill → prod
Token Comparison
MCP: ~9k
Skill: ~5k
[Test] → [Validate] → [Build]
From idea to production

Situation	Use MCP	Use Skill
Testing if something is possible	✓	
External service with official MCP server	✓	
Browser automation (no API available)	✓	
Real-time data (search, live APIs)	✓	
Repeatable workflow, same steps every time		✓
Token-sensitive production use		✓
Logic you've refined and trust		✓
Team-shared process documentation		✓

SUB-AGENTS in Claude Code

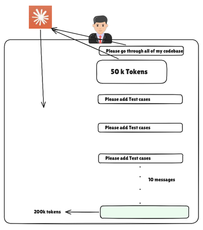
Subordinate Agents

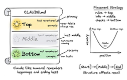
NOT A TEAM


Manager
Sales
Intern
Marketing

1. Context Overload
2. My Experience might not be required at all tasks.
3. Mangement
4. Skills
5. Cost

FREELANCER

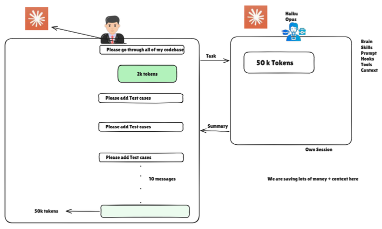

Please go through all of my codebase
50 k Tokens
Please add Test cases
Please add Test cases
Please add Test cases
10 messages
200k tokens


CLAUDE.md
Top
best remembered
example:
primacy
never delete
always use
Middle
least remembered
example:
lost middle
structure
naming
Bottom
well remembered
example:
recency
run tests
Claude (like humans) remembers beginnings and ending best
Placement Strategy
rules → top
info → middle
checks → bottom
memory
Top: high
Bottom: high
Middle: low
position
[Start] → [Middle] → [End]
Structure affects recall

In the 5 – 10 messages,

i will be incurring around 3–5 usd of cost.

And my context window as well will get fill up.

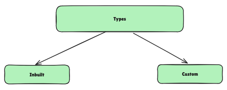

Please go through all of my codebase
2k tokens
Please add Test cases
Please add Test cases
Please add Test cases
10 messages
50k tokens
Task
Summary
Haiku
Opus
50 k Tokens
Brain
Skills
Prompt
Hooks
Tools
Context
Own Session

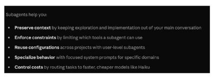
We are saving lots of money + context here

+ve

1. Context Saving or Isolation
2. Parallelism
3. Specialized Sub Agent
4. Modularity
5. Controlled Access. – We can control the tools

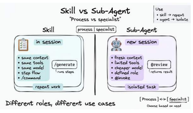
Code Exploration
General Purpose
Code Review
Writing tests
Specialized Tasks

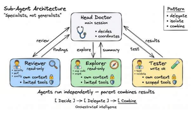

Types
Inbuilt
Custom

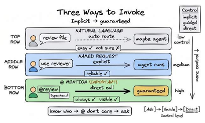


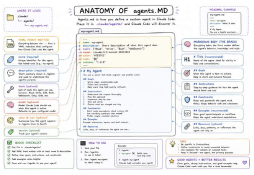
Subagents help you:

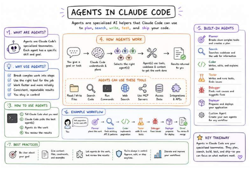
- **Preserve context** by keeping exploration and implementation out of your main conversation
- **Enforce constraints** by limiting which tools a subagent can use
- **Reuse configurations** across projects with user-level subagents
- **Specialize behavior** with focused system prompts for specific domains
- **Control costs** by routing tasks to faster, cheaper models like Haiku

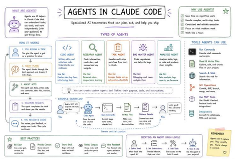

Skill vs Sub-Agent
"Process vs specialist"
Use
• skill → repeat
• agent → isolate
Skill
process | specialist
Sub-Agent
in session
• same context
• same tools
• same model
• step flow
• /command
/generate
runs steps
repeat work
new session
• fresh context
• limited tools
• cheaper model
• defined role
• @invoke
@review
returns result
isolated task
Different roles, different use cases
[Process] <-> [Specialist]
Choose based on need

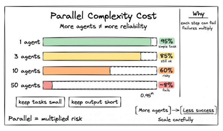

Sub-Agent Architecture
"Specialists, not generalists"
Head Doctor
main session
• decides
• coordinates
Pattern
• delegate
• isolate
• combine
review
findings
explore
summary
test
results
Reviewer
read-only
• security
• perf
• quality
• own context
• limited tools
Explorer
read-only
• map code
• orient
• own context
• limited tools
Tester
write ok
• test
• validate
• own context
• scoped tools
Agents run independently — parent combines results
[Decide] → [Delegate] → [Combine
Orchestrated intelligence


Three Ways to Invoke
Implicit → guaranteed
Control
implicit
guided
direct
TOP ROW
review file
NATURAL LANGUAGE
auto route
maybe agent
easy ✓ not sure ✗
low control
MIDDLE ROW
use reviewer
NAMED REQUEST
explicit
agent runs
reliable ✓
medium
BOTTOM ROW
@review
typeahead
@ MENTION (IMPORTANT)
direct call
guaranteed
always ✓ visible ✓
high
more control →
know who → @ don't care → ask
[Ask] → [Guide] → [Direct]
Control level


ANATOMY OF agents.MD
Agents.md is how you define a custom agent in Claude Code.
Place it in .claude/agents/ and Claude Code will discover it.
WHERE IT LIVES
MINIMAL EXAMPLE
my-agent.md
YAML FRONT MATTER
MARKDOWN BODY (THE BRAIN)
# My Agent
GOOD AGENTS = BETTER RESULTS


AGENTS IN CLAUDE CODE

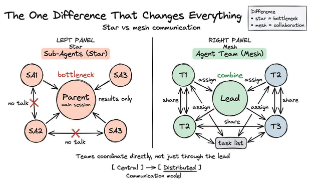

AGENTS IN CLAUDE CODE

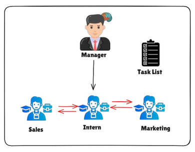


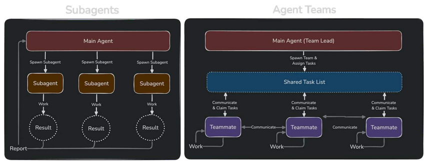
Situation	Use Sub-Agent	Just Prompt
Task needs isolated context	✅	
Task is self-contained, no back-and-forth needed	✅	
You want cheaper model for routine work	✅	
Parallel work — independent tasks	✅	
Task is small (4 lines of grep)		✅
Task requires back-and-forth with you		✅
You need agents to talk to EACH OTHER		Use Agent Teams
Two agents editing the SAME file simultaneously		❌ Never do this

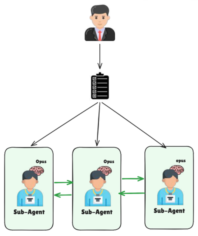
TEAMS in Claude Code

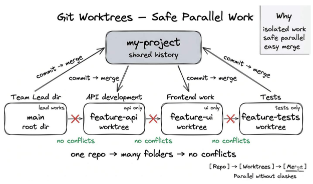
EXPERIMENTAL

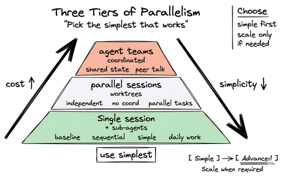

The One Difference That Changes Everything
Star vs mesh communication
Difference
• star = bottleneck
• mesh = collaboration
LEFT PANEL
Star
Sub-Agents (Star)
SA1
bottleneck
SA3
no talk
Parent
main session
results only
SA2
SA3
no talk
RIGHT PANEL
Mesh
Agent Team (Mesh)
T1
combine
assign
assign
T2
share
Lead
assign
assign
share
T2
share
T3
task list
Teams coordinate directly, not just through the lead
[Central] → [Distributed]
Communication model

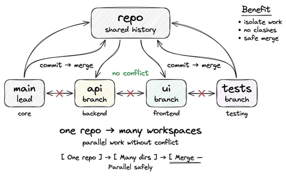

Manager
Task List
Sales
Intern
Marketing


Subagents
Main Agent
Spawn Subagent
Spawn Subagent
Spawn Subagent
Subagent
Subagent
Subagent
Work
Work
Work
Result
Result
Result
Report
Agent Teams
Main Agent (Team Lead)
Spawn Team & Assign Tasks
Shared Task List
Communicate & Claim Tasks
Communicate & Claim Tasks
Communicate & Claim Tasks
Teammate
Communicate
Teammate
Communicate
Teammate
Work
Work
Work


Opus
Sub-Agent
Opus
Sub-Agent
opus
Sub-Agent


Git Worktrees — Safe Parallel Work
Why
isolated work
safe parallel
easy merge
my-project
shared history
commit → merge
commit → merge
commit → merge
commit → merge
Team Lead dir
lead works
main
root dir
API development
api only
feature-api
worktree
Frontend work
ui only
feature-ui
worktree
Tests
tests only
feature-tests
worktree
no conflicts
no conflicts
no conflicts
one repo → many folders → no conflicts
[Repo] → [Worktrees] → [Merge]
Parallel without clashes


Three Tiers of Parallelism
"Pick the simplest that works"
Choose
simple first
scale only
if needed
agent teams
coordinated
shared state peer talk
cost ↑
parallel sessions
worktrees
independent no coord parallel tasks
simplicity ↓
Single session
+ sub-agents
baseline sequential simple daily work
use simplest
[Simple] → [Advanced]
Scale when required

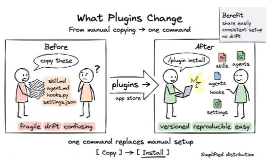

repo
shared history
Benefit
• isolate work
• no clashes
• safe merge
commit → merge
commit → merge
no conflict
main
lead
core
api
branch
backend
ui
branch
frontend
tests
branch
testing
one repo → many workspaces
parallel work without conflict
[One repo] → [Many dirs] → [Merge —
Parallel safely

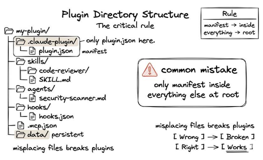
Best Cases for Agent Teams

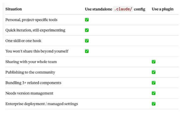
1. Large Scale Refactoring
2. Full stack Feature Dev.
3. Parallel code review + testing
4. Security Audit at scale.
5. Code base Understanding (longer)

- Sequential tasks with hard dependencies (Task B needs Task A's output)
- Same-file edits simultaneously (guaranteed conflicts)
- Small tasks under 10 minutes (overhead exceeds benefit)
- Work that needs tight coordination more efficiently done by one agent
- When experimental bugs would block you (use manual parallel instead)

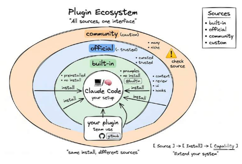
PLUGINS – CLAUDE CODE

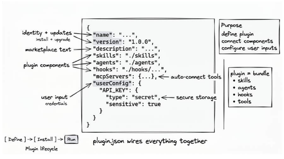

What Plugins Change
From manual copying → one command
Benefit
share easily
consistent setup
no drift
Before
copy these
skill.md
agent.md
hooks.py
settings.json
fragile
drift
confusing
plugins →
app store
After
/plugin install
skills
agents
agents
hooks
settings
versioned
reproducible
easy
one command replaces manual setup
[Copy] → [Install]
Simplified distribution


Plugin Directory Structure
The critical rule
Rule
manifest → inside
everything → root
my-plugin/
.claude-plugin/
plugin.json
skills/
code-reviewer/
SKILL.md
agents/
security-scanner.md
hooks/
hooks.json
.mcp.json
data/ persistent
only plugin.json here
common mistake
only manifest inside
everything else at root
misplacing files breaks plugins
[Wrong] → [Broken]
[Right] → [Works]
misplacing files breaks plugins

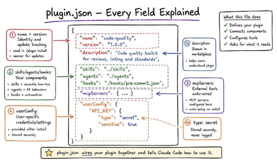


Situation	Use standalone `.claude/` config	Use a plugin
Personal, project-specific tools	✅	
Quick iteration, still experimenting	✅	
One skill or one hook	✅	
You won't share this beyond yourself	✅	
Sharing with your whole team		✅
Publishing to the community		✅
Bundling 3+ related components		✅
Needs version management		✅
Enterprise deployment / managed settings		✅


Plugin Ecosystem
"All sources, one interface"
Sources
built-in
official
community
custom
community
official
built-in
Claude Code
your setup
your plugin
check source
"same install, different sources"
[Browse] → [Install] → [Run]
"Extend your system"

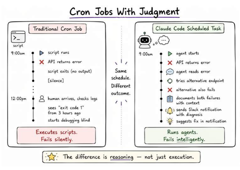

identity + updates
marketplace text
plugin components
user input
Purpose
define plugin
connect components
configure user inputs
plugin = bundle
skills
agents
hooks
tools
auto-connect tools
secure storage
[Define] → [Install] → [Run]
Plugin lifecycle
plugin.json wires everything together

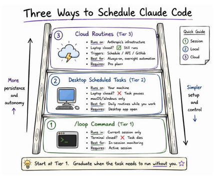
LSP – Language Server Protocol

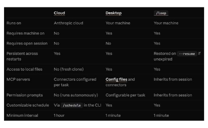

plugin.json – Every Field Explained
What this file does
"sensitive": true
Stored securely, never logged
provided after install
stored securely
plugin.json wires your plugin together and tells Claude Code how to use it.

Scheduling & Automation Loops

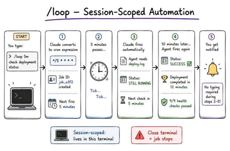
Claude Code That Works While You Sleep

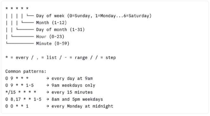

Cron Jobs With Judgment
Traditional Cron Job
script
9:00am
script runs
API returns error
script exits (no output)
[silence]
12:00pm
human arrives, checks logs
sees "exit code 1" from 3 hours ago
starts debugging blind
Executes scripts
Fails silently
Same schedule.
Different outcome.
Claude Code Scheduled Task
9:00am
agent starts
API returns error
agent reads error
tries alternative endpoint
alternative also fails
documents both failures with context
sends Slack notification with diagnosis
suggests fix in notification
Runs agents
Fails intelligently
The difference is reasoning — not just execution.


Three Ways to Schedule Claude Code
Cloud Routines (Tier 1)
Desktop Scheduled Tasks (Tier 2)
/loop Command (Tier 3)
More persistent and autonomous
Simpler setup and control



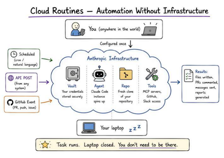
	Cloud	Desktop	`/loop`
Runs on	Anthropic cloud	Your machine	Your machine
Requires machine on	No	Yes	Yes
Requires open session	No	No	Yes
Persistent across restarts	Yes	Yes	Restored on `--resume` if unexpired
Access to local files	No (fresh clone)	Yes	Yes
MCP servers	Connectors configured per task	Config files and connectors	Inherits from session
Permission prompts	No (runs autonomously)	Configurable per task	Inherits from session
Customizable schedule	Via `/schedule` in the CLI	Yes	Yes
Minimum interval	1 hour	1 minute	1 minute

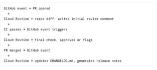

/loop — Session-Scoped Automation
You get notified
No typing required during steps 2-5!
Session-scoped
Lives in this terminal
Close terminal
= job stops



```
* * * * *
| | | | └── Day of week (0=Sunday, 1=Monday...6=Saturday)
| | | └──── Month (1-12)
| | └────── Day of month (1-31)
| └──────── Hour (0-23)
└────────── Minute (0-59)

* = every / , = list / - = range / / = step

Common patterns:
0 9 * * *       → every day at 9am
0 9 * * 1-5     → 9am weekdays only
*/15 * * * *    → every 15 minutes
0 8,17 * * 1-5  → 8am and 5pm weekdays
0 0 * * 1       → every Monday at midnight
```

Headless Mode


Cloud Routines — Automation Without Infrastructure
You (anywhere in the world)
Configured once
Scheduled
(cron / natural language)
API POST
(from any system)
GitHub Event
(PR, push, issue)
Anthropic Infrastructure
Vault
Your credentials stored securely
Agent
Claude Code instance spins up
Repo
Fresh clone of your repository
Tools
MCP servers, GitHub, Slack access
Results:
files written, PRs commented, messages sent, reports generated
Your laptop
z z z
Task runs. Laptop closed. You don't need to be there.

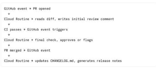


```
GitHub event → PR opened
  ↓
Cloud Routine → reads diff, writes initial review comment
  ↓
CI passes → GitHub event triggers
  ↓
Cloud Routine → final check, approves or flags
  ↓
PR merged → GitHub event
  ↓
Cloud Routine → updates CHANGELOG.md, generates release notes
```

Automated PR Workflow

"When you schedule an agent that runs without you watching, you've created a system that:"

- *Can read and write your files*
- *Can run bash commands*
- *Can call external APIs (via MCP)*
- *Can commit code and open PRs (via GitHub MCP)*
- *Can send messages (via Slack, email MCP)*



```
GitHub event → PR opened
  ↓
Cloud Routine → reads diff, writes initial review comment
  ↓
CI passes → GitHub event triggers
  ↓
Cloud Routine → final check, approves or flags
  ↓
PR merged → GitHub event
  ↓
Cloud Routine → updates CHANGELOG.md, generates release notes
```

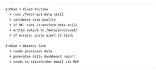
Error Response Loop

Bad: "Update the codebase to fix any linting errors"
Good: "Identify linting errors and write the list to /reports/lint-issues.md
Do NOT modify any source files — only report"

Bad: "Deploy when tests pass"

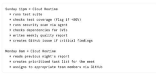
Good: "When tests pass, write a summary to /ready-to-deploy.md
and send a Slack message asking for human approval to deploy"

1. Start with Read Only
2. Write output to a specific, non destructive Path
3. Use human review gates for irreversible actions
4. Log Everything
5. Test Everything.



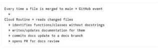
```
6:00am → Cloud Routine
  → runs /fetch-api-data skill
  → validates data quality
  → if OK: runs /transform-data skill
  → writes output to /data/processed/
  → if errors: posts alert to Slack

8:00am → Desktop Task
  → reads processed data
  → generates daily dashboard report
  → sends to stakeholder email via MCP
```

Daily Data Engineering



```
Sunday 11pm → Cloud Routine
  → runs test suite
  → checks test coverage (flag if <80%)
  → runs security scan via agent
  → checks dependencies for CVEs
  → writes weekly quality report
  → creates GitHub issue if critical findings

Monday 8am → Cloud Routine
  → reads previous night's report
  → creates prioritised task list for the week
  → assigns to appropriate team members via GitHub
```

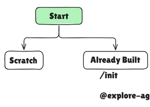
Weekly Code Quality

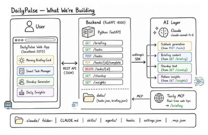


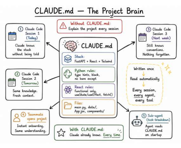
```
Every time a file is merged to main → GitHub event
  ↓
Cloud Routine → reads changed files
  → identifies functions/classes without docstrings
  → writes/updates documentation for them
  → commits docs update to a docs branch
  → opens PR for docs review
```

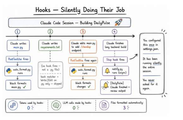
Continuous Documentation

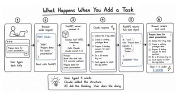
End to End Mega Project with Claude Code

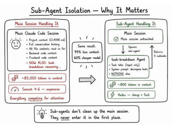

Start
Scratch
Already Built
/init
@explore-agent

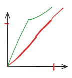

DailyPulse — What We're Building
User
DailyPulse Web App
(localhost:5173)
Morning Briefing Card
Smart Task Manager
Standup Generator
Daily Insights
REST API
(JSON)
Backend (FastAPI :8000)
Python FastAPI
GET /briefing
GET /tasks
POST /tasks
PUT /tasks/{id}/complete
DELETE /tasks/{id}
GET /standup
GET /insights
data/
(tasks.json, briefing.json)
anthropic SDK
AI Layer
Claude
claude-sonnet-4-6
Subtask generation
(from POST /tasks)
Briefing content
(from GET /briefing)
Standup text
(from GET /standup)
Pattern insights
(from GET /insights)
MCP
Tavily MCP
Real-time web tips
→ /briefing
.claude/ folder:
CLAUDE.md
skills/
agents/
hooks
settings.json
.mcp.json

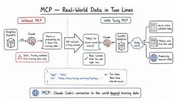

CLAUDE.md — The Project Brain

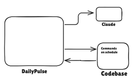

Hooks — Silently Doing Their Job

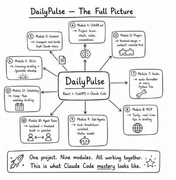

What Happens When You Add a Task


Sub-Agent Isolation — Why It Matters






Claude
Commands on schedule
DailyPulse
Codebase


DailyPulse — The Full Picture
DailyPulse
One project. Nine modules. All working together.
This is what Claude Code mastery looks like.